

GitHub Setup for DATA 3402

WSL, macOS and Linux — do this once, use it all semester

Amir Farbin

Contents

GitHub Setup for DATA 3402	1
What you are setting up	1
Step 0 — prerequisites	2
Step 1 — fork the course repository	2
Step 2 — authenticate	2
Option A — GitHub CLI (recommended)	2
Option B — SSH keys	3
Step 3 — make a course directory	3
Step 4 — clone your fork	3
Step 5 — tell git who you are	4
Step 6 — fix the remotes	4
Step 7 — your first push	4
Every week after that	5
If you get a merge conflict anyway	5
When something breaks	6
Final check	6

GitHub Setup for DATA 3402

WSL, macOS and Linux. Do this once; the same workflow carries the whole semester.

What you are setting up

You submit every lab in this course through GitHub. The arrangement is:

- You **fork** the course repository, giving you your own copy on your own account.
- You **clone your fork** onto your machine.
- You wire the clone so that **git pull pulls from the class** and **git push pushes to your fork**.
- You submit by committing your notebook and pushing.

One line to remember: **pull from class, push to my fork, never force push.**

Because the course repository is public, your fork will be public too. That is expected.

Where to get help. Bring anything that doesn't work to the Lab 4 session — that session exists partly to debug this. You are not expected to have it working perfectly on your own first.

Step 0 — prerequisites

Windows: Ubuntu installed through WSL, and a terminal that opens into Ubuntu.

macOS: Terminal, with the command line tools installed:

```
xcode-select --install
```

Linux: you already have what you need.

Everyone: a GitHub account.

Step 1 — fork the course repository

In a browser, log in to GitHub and go to the course repository:

<https://github.com/UTA-DataScience/DATA3402.Fall.2026>

Press **Fork** at the top right and fork it into your own account. You should end up looking at your own copy, at `github.com/YOUR_USERNAME/DATA3402.Fall.2026`.

Step 2 — authenticate

Passwords no longer work for git over HTTPS. Pick one of these. **Option A is recommended** and is what most of you should do.

Option A — GitHub CLI (recommended)

Works the same on WSL, macOS and Linux, and needs no keys.

1. Install the GitHub CLI:

- Ubuntu / WSL: `sudo apt install gh`
- macOS: `brew install gh`, or download the installer from <https://cli.github.com>

2. Log in:

```
gh auth login
```

Choose **GitHub.com**, then **HTTPS**, then **authenticate via browser**. Approve the login in the browser that opens — a passkey works here.

3. Check it:

```
gh auth status
```

That's it. Ordinary `git` commands now work over HTTPS.

Option B — SSH keys

More setup, and worth it if you already know SSH or expect to use several machines.

1. Generate a key:

```
ssh-keygen -t ed25519 -C "your_netid@uta.edu"
```

Press Enter at every prompt to accept the defaults.

2. Start the agent and add the key:

```
eval "$(ssh-agent -s)"  
ssh-add ~/.ssh/id_ed25519
```

3. Copy the public half:

```
cat ~/.ssh/id_ed25519.pub
```

On GitHub go to **Settings** → **SSH and GPG keys** → **New SSH key**, paste it, and save.

4. Test:

```
ssh -T git@github.com
```

It should tell you that you have authenticated.

Whichever you pick, this is a one-time setup per machine. If authentication stops working later, rerun `gh auth login` (Option A) or check that your key is still added (Option B).

Step 3 — make a course directory

```
cd ~  
mkdir -p Data-3402  
cd Data-3402
```

WSL users, this matters. Work inside your Linux home directory, as above — *not* under `/mnt/c/...` Working on the Windows filesystem from WSL is much slower and causes git and Jupyter to misbehave in ways that are hard to diagnose.

Step 4 — clone your fork

On **your fork's** GitHub page, press the green **Code** button and copy the URL. Then, matching the method you chose in Step 2:

```
# Option A (HTTPS)  
git clone https://github.com/YOUR_USERNAME/DATA3402.Fall.2026.git  
  
# Option B (SSH)  
git clone git@github.com:YOUR_USERNAME/DATA3402.Fall.2026.git  
  
cd DATA3402.Fall.2026
```

With SSH the first connection asks “Are you sure you want to continue connecting?” — answer yes.

Already cloned the class repository in Lab 1? Then you already have a DATA3402.Fall.2026 folder in ~/Data-3402, and cloning your fork there stops with *destination path already exists*. Don’t clone again, and don’t delete that folder. Copy your fork’s URL as above — Step 6 needs it — then `cd` into that folder and carry on from Step 5. Steps 5 to 7 work on that clone unchanged.

Step 5 — tell git who you are

If git complains that your **author identity is unknown**, set it:

```
git config --global user.name "Your Full Name"
git config --global user.email "your_netid@uta.edu"
git config --global --list
```

Step 6 — fix the remotes

This is the step that makes the whole arrangement work, and the one people skip.

Your clone currently points at your fork for everything. You want it to *fetch from the class* and *push to your fork*.

```
git remote -v                                # see what you have now
git remote remove origin
git remote add origin https://github.com/UTA-DataScience/DATA3402.Fall.2026.git
```

Then point pushes at your fork — the URL you cloned from:

```
# Option A (HTTPS)
git remote set-url --push origin https://github.com/YOUR_USERNAME/DATA3402.Fall.2026.git

# Option B (SSH)
git remote set-url --push origin git@github.com:YOUR_USERNAME/DATA3402.Fall.2026.git
```

Check it:

```
git remote -v
```

You want **fetch** → **the class repository** and **push** → **your fork**. If both lines say the same thing, the `set-url --push` didn’t take; run it again.

There is one remote, called **origin**, and it is deliberately lopsided. `git pull` brings you this week’s material; `git push` sends your work to your fork.

Step 7 — your first push

```
git add .
git commit -m "Initial setup"
git push
```

If git says **the current branch main has no upstream branch**:

```
git push --set-upstream origin main
```

If it says **rejected (non-fast-forward)** — which is common and expected the first time, because your fork and the class repo have separate histories:

```
git config pull.rebase false
git pull origin main --allow-unrelated-histories
git push
```

That is safe.

Every week after that

```
git pull                                # get the new material
# ... work on your notebook ...
git add .
git commit -m "Lab 3 solution"
git push                                # goes to your fork
```

Name your solution differently from the file we hand out. If the lab arrives as Lab.3.ipynb, copy it to Lab.3.solution.ipynb as soon as you pull it, and work only in the copy:

```
cp Lab.3.ipynb Lab.3.solution.ipynb
```

Copy it, don't mv it. After a rename, git follows our later changes into your renamed file, and you get the conflict anyway.

If you edit the handed-out file in place, the next `git pull` will try to merge our changes into yours and you will get a merge conflict in a Jupyter notebook — which is genuinely unpleasant, because the conflict lands in the notebook's JSON rather than in your code.

If you get a merge conflict anyway

CONFLICT from `git pull` means a file we handed out was edited in place, and we have changed it since. Nothing is lost. Don't try to fix the notebook's JSON by hand: back out of the pull, keep your work under its own name, and put our copy back. Type these one line at a time rather than pasting the block, since `cp` may stop to ask you something:

```
cd ~/Data-3402/DATA3402.Fall.2026/Labs/Lab.3  # the folder git named
git merge --abort                               # undo the half-finished pull
cp -i Lab.3.ipynb Lab.3.solution.ipynb         # your work, under its own name
git checkout origin/main -- Lab.3.ipynb        # our copy, back under our name
git add Lab.3.solution.ipynb
git commit -m "Keep my Lab 3 work in its own file"
git config pull.rebase false                    # pull merges, as in Step 7
git pull
git push
```

Use the folder and file that git named in place of `Lab.3`. If `cp` asks whether to overwrite, answer `n`: you already have a `Lab.3.solution.ipynb`, and your work is in it. If `git merge --abort` says there is no merge to abort, look at `git status`. If it mentions a rebase, run `git rebase --abort` and carry on. If it doesn't, git stopped the pull before merging anything — it said *your local changes would be overwritten* rather than `CONFLICT`, the same problem caught before you committed — so just carry on.

Don't reach for `git reset --hard` or a force push to get out of a conflict. Both throw work away. And when git offers `pull.rebase true`, pick `false`: with this course's one lopsided remote, a rebasing pull can quietly drop commits you have already pushed from your clone.

When something breaks

It says	What it means	What to do
not a git repository	you're in the wrong directory	<code>cd</code> <code>~/Data-3402/DATA3402.Fall.2026</code>
author identity unknown	name and email not set	Step 5
permission denied (publickey)	SSH key not registered	Step 2, Option B — or switch to Option A
rejected (non-fast-forward)	the remote has commits you don't	<code>git pull origin main</code> <code>--allow-unrelated-histories</code>
divergent branches	git wants a merge strategy	<code>git config pull.rebase false</code> — not <code>true</code>
Merge conflict in ...	a file we handed out was edited in place	<i>If you get a merge conflict anyway</i>
Your local changes ... would be overwritten	the same, before you committed	<i>If you get a merge conflict anyway</i>
destination path ... already exists	you cloned the class repository in Lab 1	don't clone again — see the note in Step 4
Authentication failed over HTTPS	password auth, which is gone	<code>gh auth login</code>
push went to the class repo	<code>set-url --push</code> didn't apply	redo Step 6 and check <code>git remote -v</code>

Final check

```
git status
git branch -vv
git remote -v
```

If `git remote -v` shows `fetch` pointing at `UTA-DataScience` and `push` pointing at your own account, you are set for the semester.